Language Interpreter Design

Part 1 due date: Mar 13 11:59pm

Part 2 due date: Mar 27 11:59pm

Part 3 due date: Apr 10 11:59pm

Total Points: 450

1. Overview

The goal of this project is to understand and build an interpreter for a small, OCaml-like, stack-based bytecode language. You will be implementing this interpreter in OCaml, like the previous assignments. The project is broken down into three parts. Part 1 is defined in Section 4 and is worth 100 points, Part 2 is defined in Section 5 and is worth 150 points, and Part 3 is defined in Section 6 and is worth 150 points. The project quizzes will be worth a total of 50 points. The due date for each part is available at the beginning of this handout.

This is an individual project. You will submit a file named `interpreter.ml` which contains a function, `interpreter`, with the following type signature:

```
val interpreter : string -> string -> unit
```

If your program does not match the type signature, it will not compile on Autolab and you will receive 0 points. You may, however, have helper functions defined outside of `interpreter`, the grader is only explicitly concerned with the type of `interpreter`.

1.1. Submission Guidelines

You must submit a solution for each part, and each part is graded individually. However, that doesn't mean you should create a new solution from scratch for each part; your submissions for later parts should be built on your solutions to previous parts. Late submissions will not be accepted and will be given a score of 0. Only your last Autolab submission for each part will be graded for your final grade.

Some test cases will be provided on Piazza for you to test your code locally. These will not be exhaustive, so you must write your own tests to check your interpreter against all the functionality described in this document.

After part 1 and part 2 are due, there will be an additional (zero-point) autograder assessment for them that you should submit to before working on later parts. These assessments will test all the test cases from the real assessments for previous parts. You should use these assessments to make sure your code compiles and does not break the previous parts, since later parts will inevitably exercise the functionality from previous parts. If you didn't get full marks for previous parts, the zero-point assessments for later parts are also a good chance for you to fix your code before you move on (again, later assessments will still test the functionality from previous parts).

Do not wait until the previous part's deadline passes to begin working on the next part. You can use Git branches or worktrees to checkpoint your solution for one part if you move on to the next before the previous part's deadline. Start as early as possible. Do not submit your code last minute. If you have any questions, please ask well before the due date. Also check out the FAQs in Section 7.

2. Functionality

You will be implementing an interpreter for a very simple programming language. Your program will be given a series of *commands*, and your interpreter will execute each while modifying a *stack*.

2.1. Top-Level API

You will write a function with the following signature:

```
let interpreter (input : string) (output : string) : unit = ...
```

The `input` and `output` variables are file paths that are passed in as strings, just like in the Pangram assignment (you can steal the IO-handling code from `pangram_main.ml`!). Your function should write any output your interpreter produces to the file specified by `output`.

2.2. Input Format

The input file will be a series of commands, each on its own line. Commands are things like `push 1`, `mul`, `quit`, etc. (The semantics of commands are discussed in Section 4.)

You should read the input file from top to bottom and execute each in the order it was read. We highly recommend reading all of the commands into a list prior to executing them, separating IO-handling from the actual interpretation of the

commands. The input file can be arbitrarily long. You may find the library function `String.split_on_char` and `String.trim` to be useful.

You can make the following assumptions:

- Expressions given in the input file are in correct formats. For example, there will not be expressions like `push` (which is missing an argument), `3` (which has no meaning on its own), or `add 5` (which has an extra argument). See Section 3 for the valid format.
- There will be only one operator per line of the input file.
- All test cases will have a quit statement at the end to exit your interpreter, and that "quit" will never appear mid-program.
- Your interpreter function will only be called ONCE per execution of your program.
- The input file will not be empty and all the commands in the input file will be valid commands as defined in Section 3. Commands defined in part 2 and part 3 will not appear in the input file for part 1, and commands defined in part 3 will not appear in the input file for part 2.

input	stack
push 1 quit	1

input	stack
push 6 push 2 div mul quit	`:error:` 3

input	stack
push 5 neg push 10 push 20 add quit	30 -5

input	stack
push :true: push 7 push 8 push :false: pop sub quit	-1 `:true:`

input	stack
push 10 push 2 push 8 mul add push 3 sub quit	23

Figure 1: *Some examples of input files and the stacks they should produce.*

2.3. Output Format

The output file should contain any outputs that the program produces during execution. We discuss this more in Section 4.

3. Grammar

The following is a context free grammar for the bytecode language you will be implementing. Terminal symbols are identified by `monospace font`, and nonterminal symbols are identified by *italic font*. Anything enclosed in [brackets] denotes an optional character (zero or one occurrences). The form $(set_1 \mid set_2 \mid set_n)$ means a choice of one character from any one of the n sets. A set enclosed in {braces} means zero or more occurrences.

The set *digit* is the set of digits {0,1,2,3,4,5,6,7,8,9}, *letter* is the set of all characters in the English alphabet (lowercase and uppercase), and *ASCII* is the ASCII character set. The set *simpleASCII* is *ASCII* without quotation marks and the backslash character. Do note that this necessarily implies that escape sequences will not need to be handled in your code.

3.1. Constants

$$
\begin{aligned}
const &::= int \\
&\mid bool \\
&\mid error \\
&\mid string \\
&\mid name \\
&\mid unit
\end{aligned}
$$

$$
\begin{aligned}
int &::= [-]digit\{digit\} \\
bool &::= \text{:true:} \\
&\mid \text{:false:}
\end{aligned}
$$

$$
\begin{aligned}
error &::= \text{:error:} \\
unit &::= \text{:unit:} \\
string &::= \text{"}simpleASCII\{\text{simpleASCII}\}\text{"} \\
simpleASCII &::= ASCII \setminus \{\text{'\textbackslash', '"'}\} \\
name &::= \{_\}letter\{letter \mid digit \mid _\}
\end{aligned}
$$

3.2. Programs

$$
prog ::= com\{com\}\ \text{quit}
$$

$$
\begin{aligned}
com &::= \text{push } const \\
&\mid \text{add} \\
&\mid \text{sub} \\
&\mid \text{mul} \\
&\mid \text{div} \\
&\mid \text{rem} \\
&\mid \text{neg} \\
&\mid \text{swap} \\
&\mid \text{pop} \\
&\mid \text{toString}
\end{aligned}
$$

$$
\begin{aligned}
com\ (\text{cont.}) &::= \text{cat} \\
&\mid \text{and} \\
&\mid \text{or} \\
&\mid \text{not} \\
&\mid \text{lessThan} \\
&\mid \text{equal} \\
&\mid \text{if} \\
&\mid \text{bind} \\
&\mid \text{let } com\{com\} \text{ end} \\
&\mid funBindcom\{com\}[\text{return}]\ \text{funEnd}
\end{aligned}
$$

$$
funBind ::= \text{fun } name_1 name_2
$$

`println`	`call`

4. Part 1: Basic Computation (100 Points)

Your interpreter should be able to handle the following commands. **Note**: Stacks grow from the bottom up; a stack that looks like

stack
3
2
1

first had 1 pushed onto it, then 2, and then 3. (If we were to pop something off the top of the stack, we would get 3.)

4.1. push

4.1.1. Pushing Integers to the Stack

push *num*

where *num* is an integer, possibly with a `-` suggesting a negative value. Here `-0` should be regarded as `0`. Entering this expression will simply push *num* onto the stack. For example,

input	stack
push 5	0
push -0	5

4.1.2. Pushing Strings to the Stack

push *str*

where *str* is a string literal consisting of a sequence of characters enclosed in double quotation marks, as in "this is a string". Executing this command would push the string onto the stack:

input	stack
push "deadpool"	`batman`
push "batman"	`deadpool`

Spaces are preserved in the string, i.e. any preceding or trailing whitespace must be kept inside the string that is pushed to the stack(the whitespace is represented by the symbol ␣ in the example below):

input	stack
push "deadp␣ool␣"	`this␣is␣a␣string␣␣`
push "this␣is␣a␣string␣␣"	`deadp␣ool␣`

You may assume that the string is always legal; it will not contain quotations or escape sequences within the string itself (i.e. neither double quotes nor backslashes will appear inside a string).

4.1.3. Pushing Names to the Stack

push *name*

where *name* consists of a sequence of letters, digits, and underscores, starting with a letter or underscore (see Section 3).

input	stack	input	stack
push a	13	push __name1__	3
push 13	a	push 3	__name1__

If the *name* does not conform the specification, push the error literal `:error:` onto the stack instead of pushing *name*.

In Part 2 (Section 5.7), we will see how to a value to a name using the `bind` operation.

4.1.4. Pushing Booleans to the Stack

push *bool*

There are two boolean literals: `:true:` and `:false:`. Your interpreter should push the corresponding value onto the stack. For example,

input	stack
push 5 push :true:	`:true:` 5

4.1.5. Pushing Errors and Units to the Stack

push :error:
push :unit:

Similar with boolean literals, pushing an error literal or unit literal will push `:error:` or `:unit:` onto the stack, respectively.

input	stack
push :error:	`:unit:`
push :unit:	`:unit:`
push :error:	`:error:`
push :unit:	`:unit:`
push :unit:	`:error:`

(The `:unit:` literal does nothing, similar to `()` of type `unit` in OCaml.)

4.2. pop

The command pop removes the top value from the stack. If the stack is empty, an error literal `:error:` will be pushed onto the stack. For example,

input
push 5 pop pop

→

stack
5

→

stack

→

stack
`:error:`

4.3. Binary Integer Operations

There are five commands that perform standard mathematical operations on integers: `add`, `sub`, `mul`, `div`, and `rem`.

These operators are all *binary operators*, meaning that they take two arguments as input. Each of these operations consumes the top two values on the stack and then pushes a result back to the top of the stack.

More concretely, these commands behave in the following way:

- if top two elements in the stack are integers, pop the top element, `y`, and the next element, `x`, compute a result `r` by adding, subtracting, multiplying, dividing, or computing the remainder of `x` and `y`, and push the `r` back onto the stack;
  - for `div` and `rem`, this operation might fail if `y` is 0; in this case, push both arguments back onto the stack and then push the `:error:` literal
- if either of the top two elements in the stack are not integers, push them back in the same order and push `:error:` onto the stack;
- if there is only one element in the stack, push it back and push `:error:` onto the stack;
- if the stack is empty, push `:error:` onto the stack.

For example, this is a successful use of the add command:

input
push 5 push 8 add

→

stack
8 5

→

stack
13

and this is a successful use of the div command:

input
push 5 push 8 div

→

stack
8 5

→

stack
0

The following are error cases:

input	→	stack	→	stack
push 5 add		5		`:error:` 5

input	→	stack	→	stack

push 5
push :false:
sub
		`:false:`
		5
				`:error:`
				`:false:`
				5

input	→	stack	→	stack
push 5 push 0 div		0 5		`:error:` 0 5

4.3.1. Notes

- When implementing `rem`, it may be useful to use OCaml's `mod` operator.

4.4. neg

The command neg is to calculate the negation of an integer (negation of 0 should still be 0). It is unary therefore consumes only the top element from the stack, calculate its negation and push the result back.

`:error:` will be pushed onto the stack if:

- the top element is not an integer, push the top element back and push `:error:` onto the stack
- the stack is empty, push `:error:` onto the stack

For example, this is a successful use of the neg command:

input	→	stack	→	stack
push 5 neg		5		-5

Alternately, if the value on top of the stack is not an integer, when neg is used, that value should be pushed back onto the stack as well as `:error:`. For example:

input	→	stack	→	stack
push :false: neg		`:false:`		`:error:` `:false:`

4.5. swap

The command swap interchanges the top two elements in the stack, meaning that the first element becomes the second and the second becomes the first.

`:error:` will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:` onto the stack
- the stack is empty, push `:error:` onto the stack

For example, the following non-error case:

input	→	stack	→	stack
push 5 push 8 push :false: swap		`:false:` 8 5		8 `:false:` 5

Alternately, if there is only one element in the stack when swap is used, an error will occur and `:error:` should be pushed onto the stack. Now we have two elements in the stack (5 and `:error:`), therefore the second swap will interchange the two elements:

input
push 5
swap
swap

→

stack
5

→

stack
`:error:`
5

→

stack
5
`:error:`

4.6. toString

The command toString removes the top element of the stack and converts it to a string. If the stack is empty, the error value is pushed instead. Conversions of values to strings should be done as follows:

- **Integers** shall be rendered as decimal values with no leading zeros. Negative integers should be prefixed with a `'-'`.
- **Booleans** shall be rendered as the values :true: and :false:.
- **Error** values shall be rendered as :error:
- **Unit** values shall be rendered as :unit:
- **String** values should be left unchanged (i.e. surrounding punctuation may not be added)
- **Names** shall be converted to the corresponding string, contents unchanged.
- **Closures** (once introduced in Section 6) shall be rendered as `:fun:`

4.7. println

The println command pops a string off the top of the stack and writes it, followed by a newline, to the output file that is specified as the second argument to the interpreter function. In the case that the top element on the stack is not a string, it should be returned to the stack and an `:error:` pushed. If the stack is empty, an `:error:` shall be pushed.

4.8. quit

The command quit causes the interpreter to stop.

You may assume everything's popped before quit.

4.9. Step-by-Step Example

If your interpreter reads in expressions from `inputFile`, states of the stack after each operation are shown below:

input
push 10
push 15
push 30
sub
push :true:
swap
add
pop
neg
toString
println
toString
println
toString
println
quit

First push 10 onto the stack:

stack
10

Then push 15, 30 onto the stack:

stack
30
15
10

Then sub will pop two values from the stack, calculate 15 - 30 = −15, and push −15 back:

stack
-15
10

Then push boolean literal `:true:` onto the stack:

stack
`:true:`
-15
10

Swap consumes the top two values and push them back in the opposite order:

stack
-15
`:true:`
10

Then add will pop the top two values out, which are −15 and `:true:`, then calculate their sum. Here, `:true:` is not a numeric value therefore push both of them back in the same order as well as an error literal `:error:`:

stack
`:error:`
-15
`:true:`
10

Then pop will remove the top value from the stack:

stack
-15
`:true:`
10

Then neg will pop the top value −15, calculate its negation and push the result back onto the stack:

stack
15
`:true:`
10

Then three pairs of `toString` and `println` commands each convert the top value to a string and write it to the output file. First, `toString` converts 15 to the string "15", and `println` pops and writes it. Then `toString` converts `:true:` to the string ":true:", and `println` pops and writes it. Finally, `toString` converts 10 to "10", and `println` pops and writes it. Then quit will terminate the interpreter, the `outputFile` will contain the following content:

```
15
:true:
10
```

Now go back to the example inputs and outputs given before and make sure you understand how to get those results.

5. Part 2: Variables and Scope (150 Points)

In part 2 of the interpreter you will be expanding the types of computation you will be able to perform, adding support for immutable variables and structures for expressing scope.

Hint: If you find that there's a lot of repetition in your code when implementing the commands, and you are copying and pasting a lot of code, you should consider refactoring your code to make it more modular. Think about how higher order functions such as `map`, `fold`, and `filter` abstract away common patterns of computation, and how you can use similar techniques to abstract away common patterns in your code!

5.1. cat

The cat command computes the concatenation of the top two elements in the stack and pushes the result onto the stack. The top two values of the stack x and y are popped off and the result is the string y concatenated onto x.

`:error:` will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not strings, push the elements back onto the stack, and then push `:error:`
  - Hint: Recall that names and strings are different.

For example:

input
push "hello " push "world!" cat

→

stack
`world!` `hello␣`

→

stack
`hello world!`

Consider another example:

input
push "Michael" push Scott cat

→

stack
Scott `Michael`

→

stack

`:error:`
Scott
`Michael`

Note that strings can contain spaces, punctuation marks, and other special characters. You may assume that strings only contain ASCII characters and have no escape sequences, e.g. \n and \t.

5.2. and

The command and performs the logical conjunction of the top two elements in the stack and pushes the result (a single value) onto the stack.

`:error:` will be pushed onto the stack if:

- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not booleans, push back the elements and push `:error:`

For example:

input
push :true: push :false: and

→

stack
`:false:` `:true:`

→

stack
`:false:`

Consider another example:

input
push :true: and

→

stack
`:true:`

→

stack
`:error:` `:true:`

5.3. or

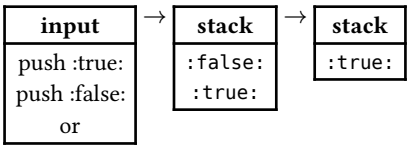
The command or performs the logical disjunction of the top two elements in the stack and pushes the result (a single value) onto the stack.

`:error:` will be pushed onto the stack if:

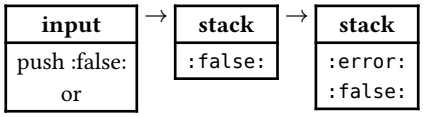
- there is only one element in the stack, push the element back and push `:error:`

- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not booleans, push back the elements and push `:error:`

For example:



Consider another example:

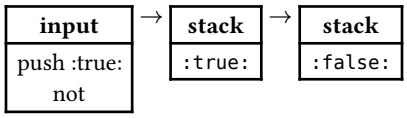


5.4. not

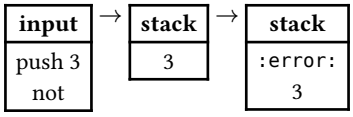
The command not performs the logical negation of the top element in the stack and pushes the result (a single value) onto the stack. Since the operator is unary, it only consumes the top value from the stack.

The `:error:` value will be pushed onto the stack if:
- the stack is empty, push `:error:` onto the stack
- if the top element is not a boolean, push back the element and push `:error:` onto the stack

For example:



Consider another example:

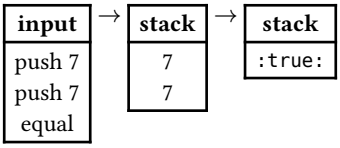


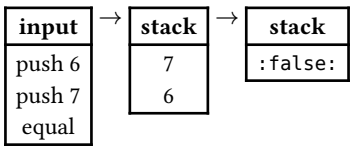
5.5. equal

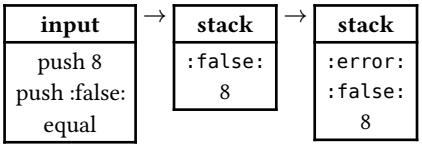
The command equal performs a numeric equality check (it does not support string comparisons). This operator consumes the top two values on the stack and pushes the result (a single boolean value) onto the stack.

The `:error:` value will be pushed onto the stack if:
- there is only one element in the stack, push the element back and push `:error:` onto the stack
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements are not integers, push back the elements and push `:error:` onto the stack

Here are a few examples:







5.6. lessThan

The command lessThan refers to numeric less than ordering. This operator consumes the top two values on the stack and pushes the result (a single boolean value) onto the stack.

The `:error:` value will be pushed onto the stack if:
- there is only one element in the stack, push the element back and push `:error:`
- the stack is empty, push `:error:` onto the stack
- if either of the top two elements aren't integers, push back the elements and push `:error:`

Note that the ordering here is the same as other binary operators. We pop `y` and then `x` and compute `x < y`.

For example:

input
push 7
push 8
lessThan

→

stack
8
7

→

stack
`:true:`

5.7. bind

Hint: Finish and test the commands above before you start working on bind.

The bind command binds a name to a value. It is evaluated by popping two values from the stack. The second value popped must be a name (see Section 4.1.3 for details on what constitutes a `name`). The name is bound to the value (the first thing popped off the stack). The value can be any of the following:
- An integer
- A string
- A boolean
- `:unit:`
- The *value* of a name that has been previously bound

The name value binding is stored in an environment data structure. The result of a bind operation is `:unit:` which is pushed onto the stack. The value `:error:` will be pushed onto the stack if:
- we are trying to bind an identifier to an unbound identifier, in which case all elements popped must be pushed back before pushing `:error:` onto the stack.
- the stack is empty, push `:error:` onto the stack.

5.7.1. Example 1

input
push a
push 3
bind

→

stack
3
a

→

stack
`:unit:`

5.7.2. Example 2

input
push sum1
push 7
bind
push sum2
push 5
bind

→

stack
7
sum1

→

stack
`:unit:`

→

stack
5
sum2
`:unit:`

→

stack
`:unit:`
`:unit:`

You can use bindings to hold values which could be later retrieved and used by functionalities you already implemented. For instance, in the example below, an addition on *a* and *name1* would add 13 + 3 and push the result 16 onto the stack.

This, in effect, allows names to be in place of proper constants in all the operations we've seen so far. Take for example, when you encounter a name in an add operation, you should retrieve the value the name is bound to, if any. Then if the value the name is bound to has the proper type, you can perform the operation.

5.7.3. Example 3

input
push a
push 13
bind
push name1
push 3
bind
push a
push name1
add

→

stack
13
a

→

stack
`:unit:`

→

stack
3
name1
`:unit:`

→

stack
`:unit:`
`:unit:`

→

stack
name1
a
`:unit:`
`:unit:`

→

stack
16
`:unit:`
`:unit:`

Notice how we can substitute a constant for a bound name and the commands work as we expect. The idea is that when we encounter names in a command, we resolve the name to the value it's bound to, and then use that value in the operation.

5.7.4. Example 4

input
push a
push 5
bind
pop
push a
push 3
add
push b
push "str"
bind
pop
push 10
push b
sub

→

stack
`:error:`
b
10
8

You can see that the add operation completes, because a is bound to an integer (5, specifically). The sub operation fails because b is bound to a string, and thus does not type check. While performing operations, if a name has no binding or it evaluates to an improper type, push `:error:` onto the stack, in which case all elements popped must be pushed back in the same order before pushing `:error:`.

5.7.5. Example 5

Bindings can be overwritten, for instance:

input
push a
push 9
bind
push a
push 10
bind

Here, the second bind updates the value of a to 10.

5.7.6. Example 6

input
push a
push 17
add

→

stack
17
a

→

stack
`:error:`
17
a

The error is because we are trying to perform an addition on an unbound variable a.

5.7.7. Common Questions

1. What values can __name__ be bound to?

   A name can be bound to integers, booleans, strings, `:unit:`, or the value of another name that has been previously bound. For example,

input
push a
push :true:
bind

   would bind *a* to `:true:`.

input
push a
push 7.5
bind

   would result in bind producing an `:error:` because *a* cannot be bound to an `:error:`.

input
push b
let
push a
push 7
bind
end
bind

   would bind *a* to 7 and then bind *b* to `:unit:` (We will discuss `let` in Section 5.9).

input
push b
push 8
bind
push a
push b
bind

   would bind *b* to 8 and then bind *a* to the **VALUE** of *b*, (what is the value of *b*?).

input
push b
push a
bind

   would result in an `:error:` because *a* is unbound when we try to bind *b* to it.

2. How can we bind identifiers to previously bound values?

input
push a
push 7
bind
push b
push a
bind

   The first bind binds the value of *a* to 7. The second bind statement would result in the name *b* getting bound to the **VALUE** of *a* which is 7. This is how we can bind identifiers to previously bound values. Note that **we are not binding *b* to *a***, we are binding it to the **VALUE** of *a*.

3. Can we have something like this?

input
push a
push 15
push a

Yes. In this case *a* is not bound to any value yet, and the stack contains:

stack
a
15
a

If we had:

input
push a
push 15
bind
push a

Then the stack would contain:

stack
a
`:unit:`

4. Can we push the same __name__ twice to the stack? For instance, what would be the result of the following:

input
push a
push a
quit

This would result in the following output file:

stack
a
a

So yes, you can push the same name twice onto the stack. Consider binding in this way:

input
push a
push a
push 2
bind

This would result in the following stack:

stack
`:unit:`
a

`:unit:` is the result of the bind operation, and the `a` is the result of pushing the first *a* onto the stack.

5.8. if

The if command pops three values off the stack: x, y and z. The third value popped (z, in this case) must always be a boolean. If z is `:true:`, executing the if command will push x back onto the stack, and if z is `:false:`, executing the if will push y back onto the stack.

`:error:` will be pushed onto the stack if:

- the third value is not a boolean, all elements (x, y, and z) should be pushed back onto the stack before pushing `:error:` onto the stack.
- the stack is empty, push `:error:` onto the stack
- there are less than 3 values on the stack, in which case all elements popped must be pushed back before pushing `:error:` onto the stack.

For example:

input
push :true:
push 9
push 8
if

→

stack
8
9
`:true:`

→

stack
8

5.8.1. Common Questions

1. What values can `if` take?

   The result of executing a `if` command can be any stack value. For instance:

input
push :true:
push "Stout"
push "Ryza"
if

   the result of the `if` would be Ryza.

input
push :false:
let
push a
push 8
bind
end
push 7
if

   the result of the `if` would be `:unit:`.

2. What is the result of executing the following:

input
push a
push 5
bind
pop
push :true:
push 4
push a
if

   The stack would have *a*. Although the value of *a* is bound to 5, we only resolve the name to the value if we need to perform computation. (For `if`, the only value needed for computation is a boolean.)

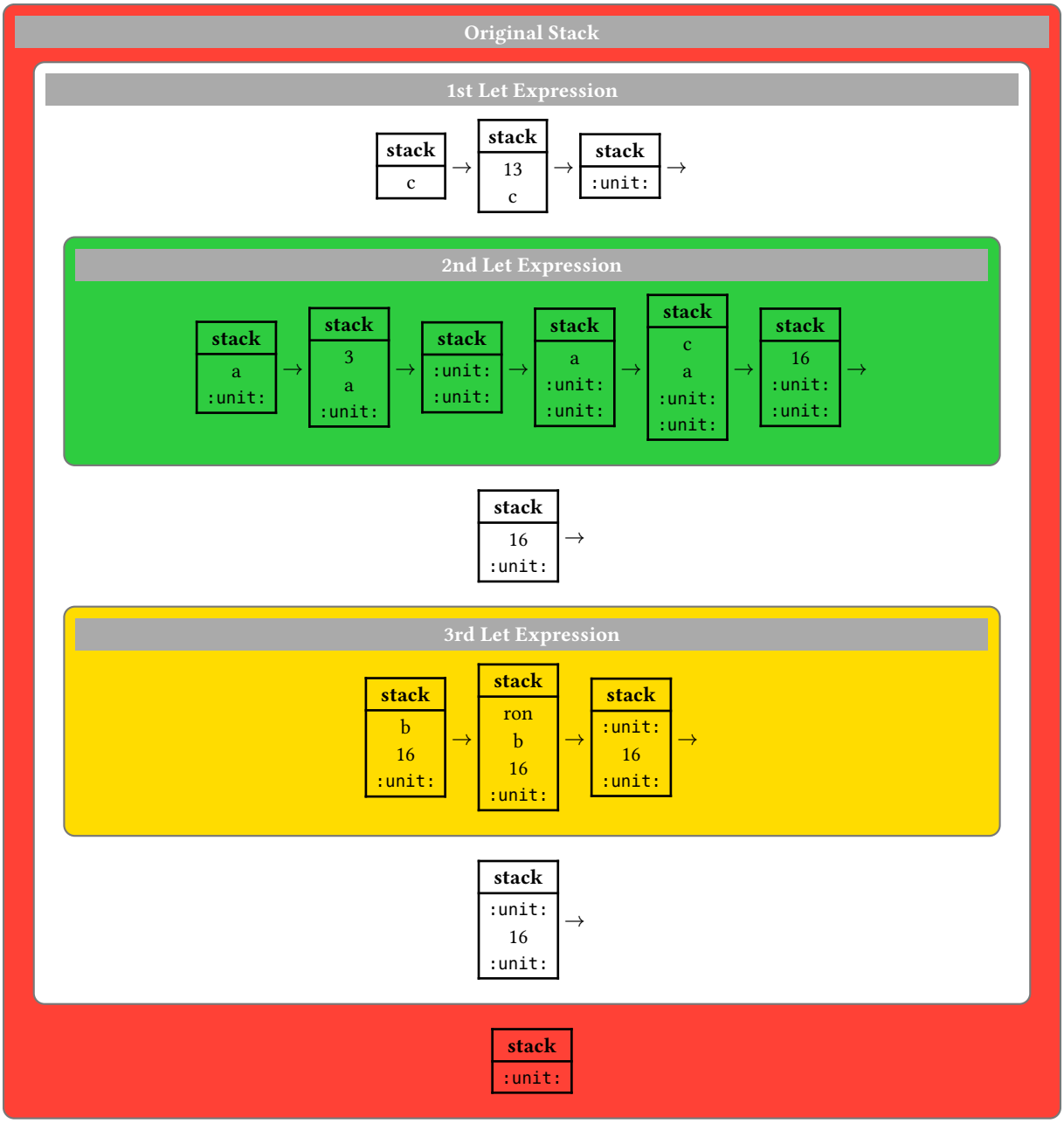
5.9. let...end

Hint: Finish and test the commands (especially **bind**) above before you start working on `let...end`!

`let...end` limits the scope of variables. "let" marks the beginning of a new environment, which is basically a sequence of bindings. You may assume that all let commands will have a matching end. The result of the `let...end` is the last stack item of the let. You may assume that the stack is left with at least 1 item after the execution of any `let...end` block. `let...end` can contain any number of operations but it will always result in a strictly larger stack. In other words, the commands in the `let...end` block will never touch the stack prior to the let. Trying to access an element that is not in scope of the `let...end` block would push `:error:` on the stack. `let...end` blocks can also be nested.

For example:

input
let
push c
push 13
bind
let
push a
push 3
bind
push a
push c
add
end
let
push b
push "ron"
bind
end
end



In the above example, the first let statement creates an empty environment (environment 1), then the name *c* is bound to 13. The result of this bind is a `:unit:` on the stack and a name value pair in the environment. The second let statement creates a second empty environment. Name *a* is bound here. To add *a* and *c*, these names are first looked up for their values in the current environment. If the value isn't found in the current environment, it is searched in the outer environment. Here, *c* is found from environment 1. The sum is pushed to the stack. A third environment is created with one binding *b*. The second last end is to end the scope of environment 3 and the last end statement is to end the scope of environment 1.

5.9.1. Common Questions

1. What is the result of executing the following code?

input
push 1
let
push 2
push 3
push 4
end
push 5

This would result in the stack:

stack
5
4
1

After the `let...end` is executed, the last item is returned, which is why we have 4 on the stack.

2. What would be the result of executing the following code?

input
let
push 3
push 7
end
push 5
add

→

stack
3

→

stack
7
3

→

stack
7

→

stack
5
7

→

stack
12

First push 3 and 7 onto the stack, then end will return 7 to the stack. Then push 5, and add will add 5 and 7 together and push the result 12 onto the stack.

3. What would be the result of executing the following code?

input
let
push a1
push 7.2
bind
end

→

stack
a1

→

stack
`:error:`
a1

→

stack
`:error:`
`:error:`
a1

→

stack
`:error:`

7.2 can't be pushed to the stack and a1 cannot be bound to `:error:` so, the result of bind would be `:error:`, and this error is returned to the previous stack after the end statement is executed.

4. What would be the result of executing the following code?

input
let
push 3
push 10
end
add

The result of the stack would be:

stack
`:error:`
10

6. Part 3: Functions (150 Points)

6.1. Function Declaration

`fun` *name1 name2*

denotes a function declaration, i.e. the start of a function called *name1*, which has one argument *name2*. The expressions that follow comprise the function body. The function body is terminated with a special keyword `funEnd`. Note, *name1* and *name2* can be any valid name, but will never be any of the keywords in our language (e.g. `add`, `push`, `pop`, `fun`, `funEnd`, etc.). Also the function name and argument name cannot be the same.

`funEnd`

denotes the end of a function body.

When the interpreter encounters a function declaration expression it should construct a closure. A closure will consist of (1) an environment, (2) the code for the function (the expressions between the function declaration and `funEnd`), and (3) the name of the argument. The value `:unit:` should be pushed to the stack once the function declaration is evaluated and the closure created and bound to the function name in the environment.

1. The environment for the closure will be a copy of the current environment. (Challenge: if you would like to optimize your closure representation you do not need the entire environment, just the bindings of the variables used inside the function that are not defined inside the function and are not the argument).
2. To compute the code for the function, you should copy all the expressions in order starting with the first expressions after the function declaration up to, but not including, the `funEnd`.
3. In the current environment you should create a binding between the function name and its closure.

6.2. call

push *funName*
push *arg*
`call`

denotes applying the function *funName* to the actual argument *arg*. When `call` is evaluated, it will apply the function *funName* to *arg* and pop both *funName* and *arg* from the stack. *arg* can either be a name (this includes function names), an integer, a string, a boolean, or `:unit:`.

When a function is called, you should first check to see if there is a binding in the current environment which maps *funName* to a closure. If one does not exist, push `:error:` onto the stack. You should then check to see if the current environment contains a binding for *arg* if it is a name instead of a value. If it does not, then you should push `:error:` onto the stack. If *arg* is an `:error:` you should push `:error:` onto the stack.

If both *funName* and *arg* have appropriate bindings, or *arg* is a valid value, then the call to the function can proceed. To do this, save the current environment and replace it with the one stored in the closure. To this environment add a binding between the argument name[1] and the value of the actual argument. Note that if *arg* is a name, then it must have a binding in the environment at the point of the call.[2] You should then save the current stack and create a new stack that will be used for the execution of the function.[3] Next retrieve the code for the function and begin executing the expressions. The function completes once the last expression in the code for the function is executed. When this happens, you should restore the environment to the environment that existed prior to the function call.[4] The stack should also be restored to what the stack was at the point of the call.[5] Once the environment has been restored, execution should resume with the expression that follows the `call`.

6.3. return

Functions can return values by using a return expression. Since functions themselves are values (a closure), functions can take other functions as arguments and can return functions. When a return expression is evaluated, the function stops execution. When this happens you should restore the environment to the environment that existed prior to the function call, just like if the function completed by executing the last expression in the function's code. The stack should also be restored to what the stack was at the point of the call. Additionally, you should push the last stack item the function pushed onto the restored stack—i.e., the top of the stack at the point of the call.

[1] you will extract the argument name from the closure

[2] this is the current environment before you push the closure's environment

[3] hint: one approach is to implement the environment as a stack of environments and to implement the stack as a stack of stacks to handle nested function calls and recursion. Other approaches exist and could be simpler

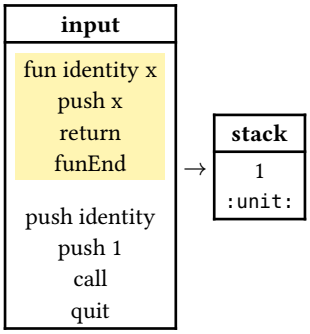
[4] hint: if you are implementing your environment as a stack of local environments, this will entail popping off the top environment

[5] hint: if you implemented your stack as a stack of stacks, this only requires popping off the top stack to restore the stack to what it was prior to the call

Please note that background color and indentation is used only to improve readability. Closure code is shown within a colored background.

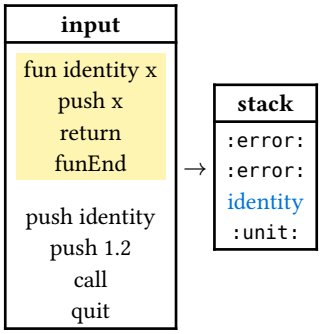
6.4. Examples

6.4.1. Example 1



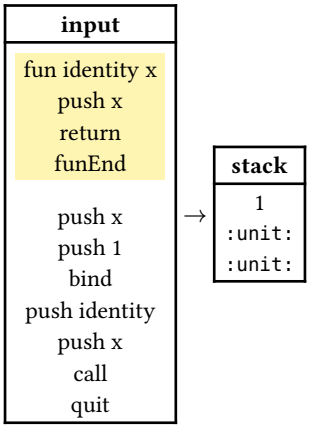
- 1 ⟶ return value of calling `identity` and passing 1 as an argument
- `:unit:` ⟶ result of declaring `identity`

6.4.2. Example 2



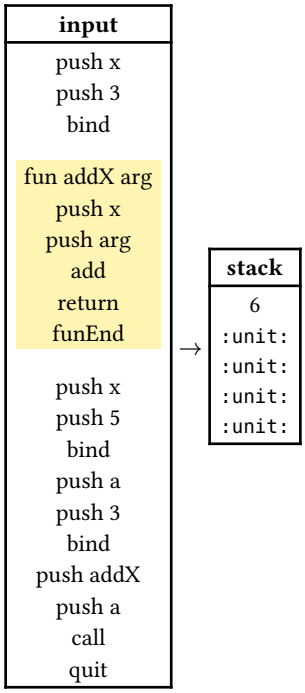
- `:error:` ⟶ error as a result of calling a function with `:error:` as the actual parameter
- `:error:` ⟶ result of pushing 1.2
- `identity` ⟶ push of identity
- `:unit:` ⟶ result of declaring `identity`

6.4.3. Example 3



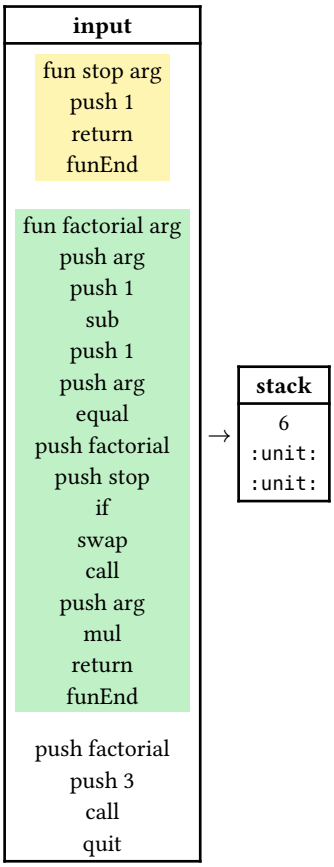
- 1 ⟶ return value of calling `identity` and passing x as an argument
- `:unit:` ⟶ result of binding x
- `:unit:` ⟶ result of declaring `identity`

6.4.4. Example 4



- 6 ⟶ result of function call
- `:unit:` ⟶ result of third binding
- `:unit:` ⟶ result of second binding
- `:unit:` ⟶ result of function declaration
- `:unit:` ⟶ result of first binding

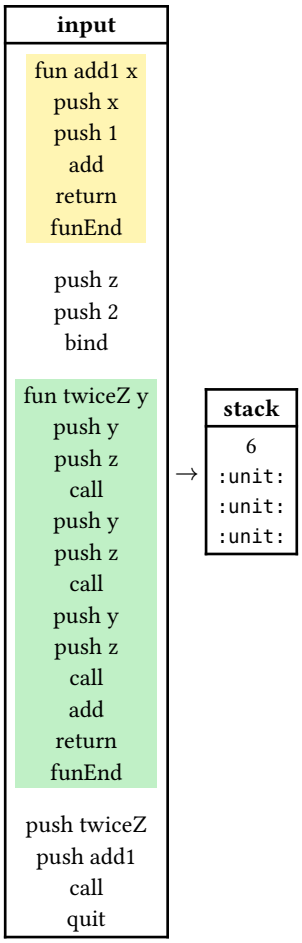
6.4.5. Example 5



- 6 ⟶ value returned from `factorial`

- `:unit:` ⟶ declaration of `factorial`
- `:unit:` ⟶ declaration of `stop`

6.4.6. Example 6



- 6 ⟶ return of calling `twiceZ` and passing `add1` as an argument
- `:unit:` ⟶ declaration of `twiceZ`
- `:unit:` ⟶ binding of `z`
- `:unit:` ⟶ declaration of the `add1` function

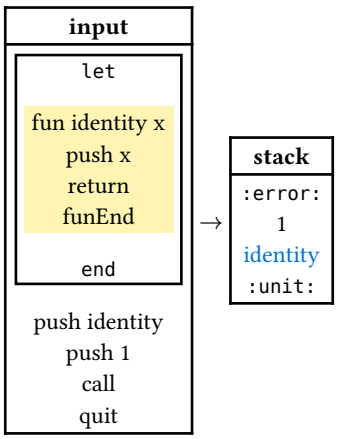
6.5. Functions and Let

Functions can be declared inside a `let` expression. Much like the lifetime of a variable binding, the binding of a function obeys the same rules. If your `let` implementation introduces a stack of environments, the closure should also take this into account.[6] The easiest way to implement this is for the closure to store the stack of environments present at the declaration of the function. (Note: you can create a more optimal implementation by only storing the bindings of the free variables used in the function—to do this you would look up each free variable in the current environment and add a binding from the free variable to the value in the environment stored in the closure)

Please note background color is used only to improve readability.

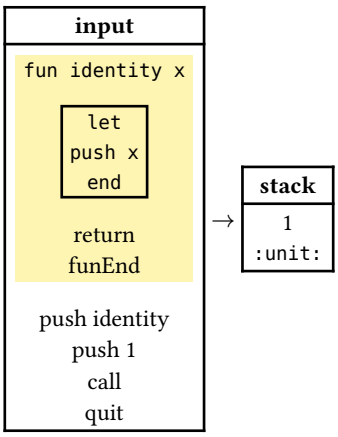
[6] A stack of environments is not the only way to handle scoping; think about what other solutions you can come up with

6.5.1. Example 1



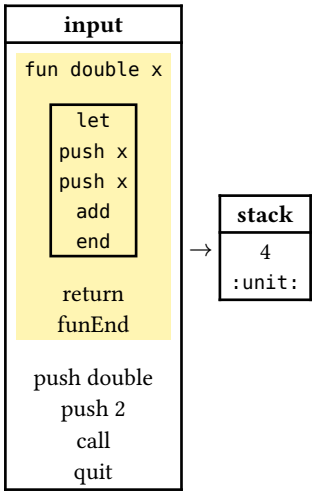
- `:error:` ⟶ error since `identity` is not bound in the environment
- 1 ⟶ push of 1
- `identity` ⟶ push of identity
- `:unit:` ⟶ result of declaring `identity`, this is the result of the `let` expression

6.5.2. Example 2



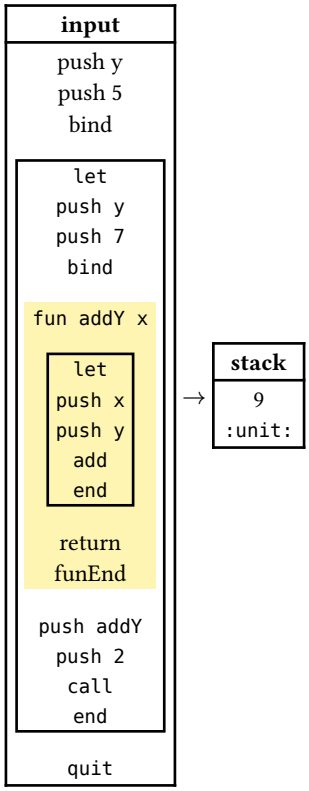
- 1 ⟶ return value of calling `identity` and passing in x as an argument
- `:unit:` ⟶ result of declaring `identity`

6.5.3. Example 3



- 4 ⟶ return value of calling `double` and passing in 2 as an argument
- `:unit:` ⟶ result of declaring `double`

6.5.4. Example 4



- 9 ⟶ return value of calling `addY` and passing in 2 as an argument
- `:unit:` ⟶ result of binding `y` to 5

6.6. First-Class Functions

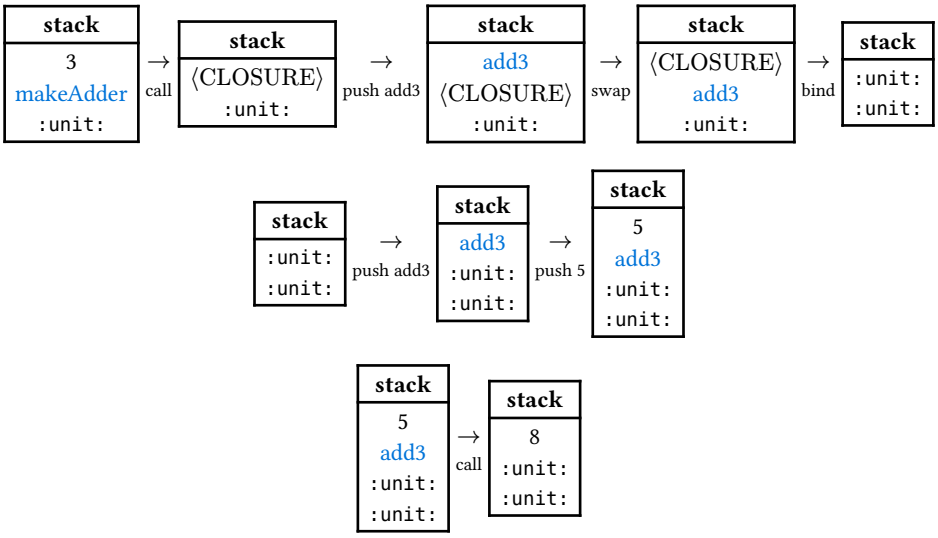
This language treats functions like any other value. They can be used as arguments to functions, and can be returned from functions.

6.6.1. Example 1: Curried Adder

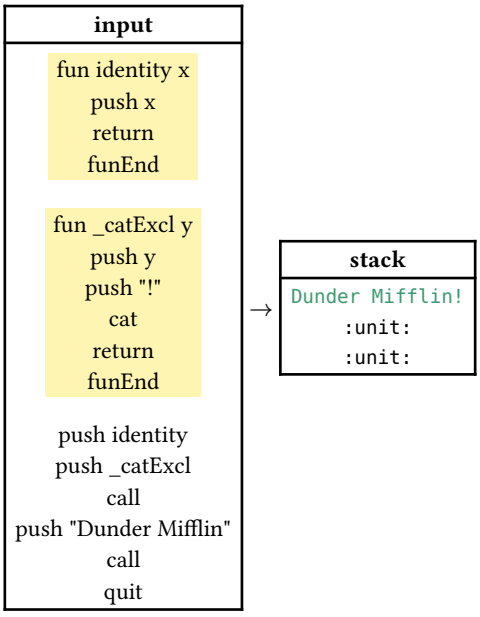
input	stack
`fun makeAdder x` fun adder y push x push y add return funEnd push adder return funEnd push makeAdder push 3 call push add3 swap bind push add3 push 5 call quit	8 `:unit:` `:unit:`

- 8 ⟶ Evaluated from calling the generated function `add3` with argument 5
- `:unit:` ⟶ The result of binding the generated function to the name `add3`
- `:unit:` ⟶ The result of declaring the function `makeAdder`

Step by step (after declaring `makeAdder`, pushing `makeAdder`, and pushing 3):

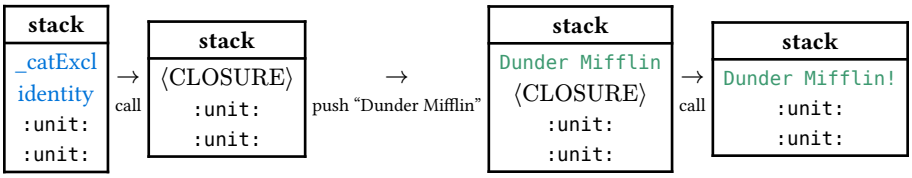


If a function is returned from another function, it need not be bound to a name in the environment it is returned in. For example:



- Dunder Mifflin! ⟶ Computed from calling the closure returned by the `identity` function applied to `_catExcl` with the argument "Dunder Mifflin".
- `:unit:` ⟶ The result of declaring the function `_catExcl`.
- `:unit:` ⟶ The result of declaring the `identity` function.

Here is a closer look at how the stack develops through this program. Note that function closures will never be on the stack when the program finishes execution.



7. Frequently Asked Questions

1. **Q: What are the contents of test case *X*?**

   A: We purposefully withhold some test cases to encourage you to write your own test cases and reason about your code. You cannot test *every* possible input into the program for correctness. We will provide high-level overviews of the test cases, but beyond that we expect you to figure out the functionalities that are not checked with the tests we provide. But you can (and should) run the examples shown in this document! They're useful on their own, and can act as a springboard to other test cases.

2. **Q: Why does my program run locally but fail on Autolab?**

   A: Check the following:

   - Ensure that your program matches the types and function header defined in section 2 on page 1.
   - Make sure that any testing code is either removed or commented out. If your program calls interpreter with input `"input.txt"`, you will likely throw an exception and get no points.
   - *Do not submit testing code.*
   - If you submit a zip or tar file, the main interpreter file must be at the top level of the archive. When extracted, there must be a file named `interpreter.ml` in the current directory.
   - `stdout` and `stderr` streams are not graded. Your program must write to the output file specified by *outputFile* for you to receive points.
   - *Close your input and output files.*
   - External libraries (e.g., Core) are not available.

3. **Q: Why doesn't Autolab give useful feedback?**

   A: Autolab is strictly a grading tool to tell you how many test cases you passed and your total score. Test and debug your program locally before submitting to Autolab. The only worthwhile feedback Autolab gives is whether or not your program compiled properly.

4. **Q: Are there any runtime complexity requirements?**

   A: Although having a reasonable runtime and space complexity is important, the only official requirement is that your program runs the test suite in less than three minutes.

5. **Q: Will the autograder run the full set of test cases?**

   A: No. As with the homework assignments, the full suite of test cases will only be run after the deadline. You should expect the cases we test pre-deadline to be similar to the test cases provided to you in this document.

6. **Q: Is my final score the highest score I received of all my submissions?**

   A: No. Your final score is only your most recent submission.

7. **Q: What can I do if an old submission received a better grade than my most recent submission?**

   A: You can always download any of your previous submissions. If the deadline is approaching, we suggest resubmitting your highest-scoring submission before Autolab locks (assuming you have not used up all your attempts yet).

8. Project Quiz (50 Points)

The week after you submit each part of the project, there will be a quiz in recitation about it. The quiz will include conceptual questions about the topic of the assignment as well as questions about *how you specifically approached the task*. This quiz will count for part of your grade on the assignment.

All of this means that it is important that you know how you solved this programming problem! We highly recommend reading over your solution before you submit to make sure you understand how it works.